AWS VPC Peering

AWS VPC Peering Report

1. Introduction and Objectives

Project Brief

The objective of this project is to build a working private network connection between two separate Virtual Private Clouds (VPCs) and demonstrate the necessity of each route and security rule. The scenario involves an application in VPC A and a second workload in VPC B. Both application servers must remain entirely private.

Objectives

- Connect two VPCs and prove private connectivity using VPC Peering so they can exchange traffic using their private IPv4 addresses.
- Establish a public bastion host in VPC A strictly for administration purposes.
- Plan and configure non-overlapping VPC and subnet CIDR blocks.
- Create public and private subnets with explicit custom route table associations.
- Activate the peering connection, correctly configure return routes, and securely limit inbound traffic using Security Groups.
- Demonstrate connectivity through a baseline failed test (before peering), successful tests (after peering and routing), and a controlled isolation failure.

2. Strategy and Implementation

Architecture Strategy

The network is designed with one internet entry point and two private workloads, bridged by a direct VPC peering connection. Administration traffic flows from a local machine through an Internet Gateway into the Bastion host via SSH Proxy Jump. Project traffic flows directly between the private instances over the peering connection using private IP addresses. Peering does not provide VPC B with internet access through VPC A.

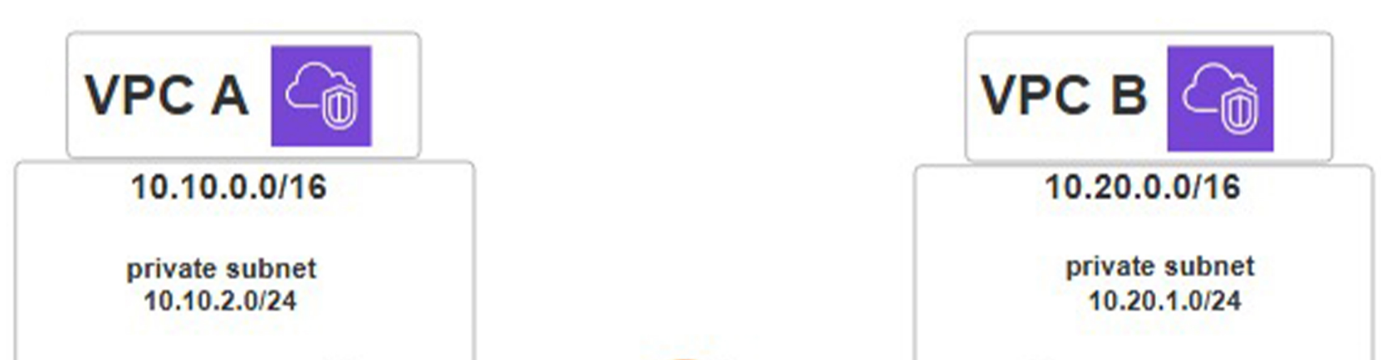
Resource and IP Address Plan

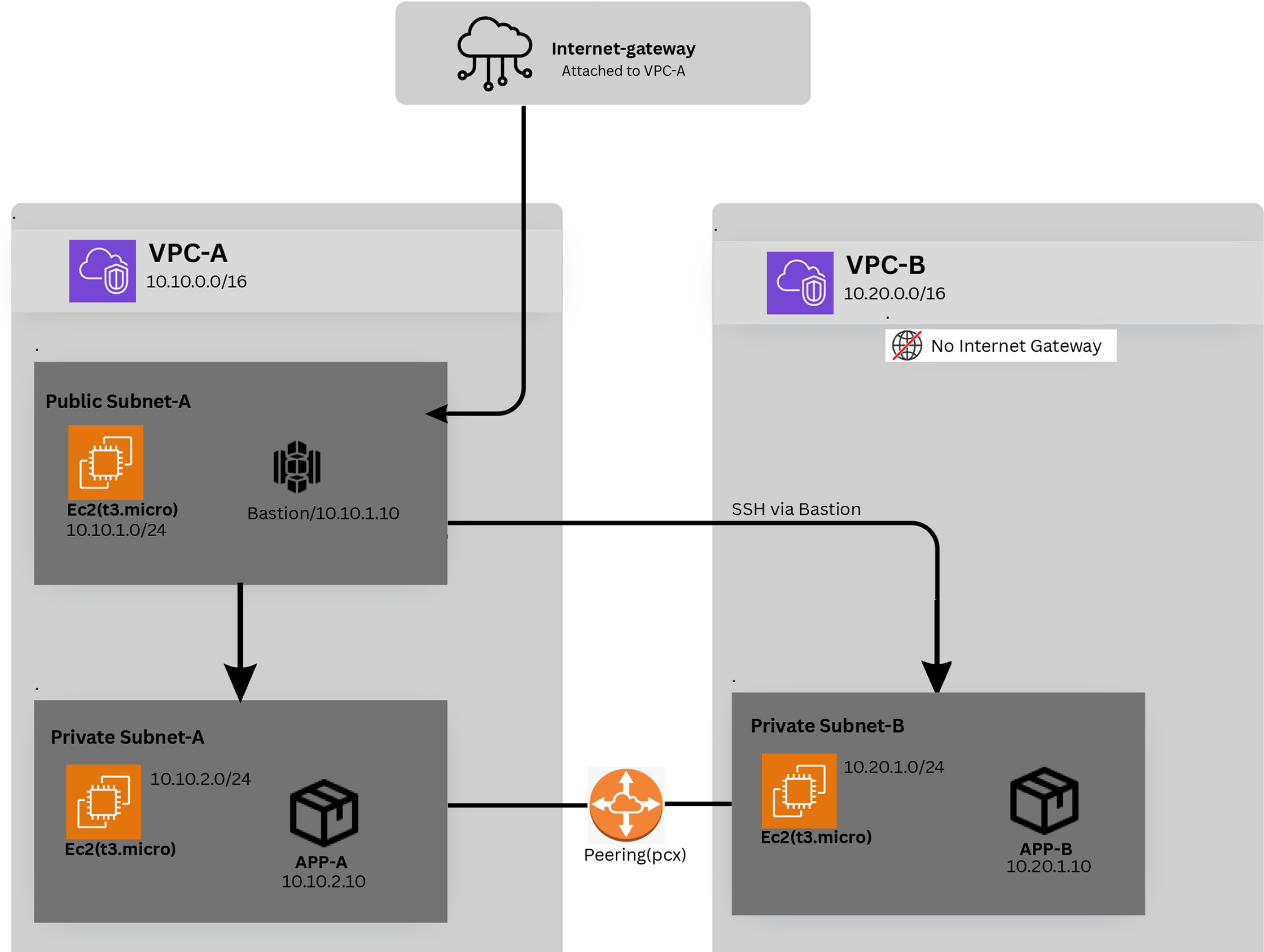
To ensure successful peering, the VPCs utilize non-overlapping CIDR blocks.

Resource	Name	CIDR Block
VPC A	vpc-a	10.10.0.0/16
Public Subnet	a-public	10.10.1.0/24
Private Subnet	a-private	10.10.2.0/24
VPC B	vpc-b	10.20.0.0/16
Private Subnet	b-private	10.20.1.0/24

Compute Components

Instance	Subnet	Private IP	Public IP
Bastion Host (bastion)	a-public	10.10.1.10	Dynamically assigned public IPv4
App-A (app-a)	a-private	10.10.2.10	None
App-B (app-b)	b-private	10.20.1.10	None





3. Roadmap + Steps

Step 1: Build the Network

- **Create VPC A:** In the AWS Console, navigate to the VPC Dashboard. Click on Create VPC. Select the "VPC only" option, enter the name **vpc-a**, and set the IPv4 CIDR block to **10.10.0.0/16**. Click Create VPC.
- **Create VPC A Subnets:** Navigate to Subnets in the left menu. Click Create subnet, select vpc-a, name it **a-public**, set the IPv4 CIDR block to **10.10.1.0/24**, and click Create subnet. Repeat this process to create a second subnet named **a-private** in vpc-a with the CIDR block **10.10.2.0/24**.
- **Internet Gateway:** Navigate to Internet gateways, click Create internet gateway, name it **igw-a**, and create it. Select it, click Actions, choose Attach to VPC, and select vpc-a.
- **Public Route Table:** Navigate to Route tables, click Create route table, name it **rt-a-public**, select vpc-a, and click Create. Go to the Subnet associations tab, click Edit subnet associations, select a-public, and save. Go to the Routes tab, click Edit routes, add a route with the destination 0.0.0.0/0 targeting the Internet Gateway (igw-a), and save.
- **Private Route Table A:** Create another route table named **rt-a-private** for vpc-a. Associate it with the a-private subnet, leaving only the default local route in the Routes tab.
- **Create VPC B & Subnet:** Repeat the VPC creation process to create **vpc-b** with CIDR 10.20.0.0/16. Create a subnet named **b-private** with CIDR 10.20.1.0/24. Create a route table named **rt-b-private**, associate it explicitly with b-private, and leave only the default local route.

Step 2: Network and Security Configuration

- **Bastion Security Group:** Navigate to Security Groups and click Create security group. Name it **sg-bastion**, select vpc-a, and add an Inbound Rule: Type SSH (TCP 22), Source My IP or your specific administrator public IPv4 address (/32). Click Create security group.
- **App-A Security Group:** Create a security group named **sg-app-a** in vpc-a. Add Inbound Rule 1: Type SSH, Source Custom, and enter the bastion's private IP 10.10.1.10/32. Add Inbound Rule 2: Type Custom ICMP - IPv4, select Echo Request, and set Source to App-B's IP 10.20.1.10/32. Click Create.
- **App-B Security Group:** Create a security group named **sg-app-b** in vpc-b. Add Inbound Rule 1: Type SSH, Source 10.10.1.10/32. Add Inbound Rule 2: Type Custom ICMP - IPv4, select Echo Request, and set Source to 10.10.2.10/32. Click Create.

Note: Default allow-all outbound rules are retained for all security groups.

Security Group	VPC	Protocol	Source
sg-bastion	vpc-a	SSH (TCP 22)	Admin public IPv4 /32
sg-app-a	vpc-a	SSH	10.10.1.10/32
sg-app-a	vpc-a	ICMP Echo Request	10.20.1.10/32
sg-app-b	vpc-b	SSH	10.10.1.10/32
sg-app-b	vpc-b	ICMP Echo Request	10.10.2.10/32

Step 3: Compute Setup

- **Key Pair:** Navigate to the EC2 Dashboard. Under Key Pairs, click Create key pair. Name it **lab-key**, select RSA format, and download it to your local machine.
- **Launch Instances:** Navigate to Instances and click Launch instances. You will launch three Amazon Linux 2023 t3.micro instances:
- **Bastion:** Name it bastion. Under Network settings, select vpc-a, subnet a-public, Enable Auto-assign Public IP, specify the private IP 10.10.1.10 under Advanced network interfaces, attach sg-bastion, and launch using lab-key.
- **App-A:** Name it app-a. Select vpc-a, subnet a-private, Disable Auto-assign Public IP, specify the private IP 10.10.2.10, attach sg-app-a, and launch using lab-key.
- **App-B:** Name it app-b. Select vpc-b, subnet b-private, Disable Auto-assign Public IP, specify the private IP 10.20.1.10, attach sg-app-b, and launch using lab-key.

Step 4: Administration and Baseline

- **Configure SSH:** On your local machine, configure your ~/.ssh/config file to use the Bastion host as a ProxyJump for connecting to the private instances.
- **Baseline Test:** Run a baseline test by SSH-ing into App-A and pinging App-B (`ping 10.20.1.10`) to confirm the expected connectivity failure before peering is established.

Step 5: Establish Peering and Routing

- **Create Peering Connection:** In the VPC Dashboard, navigate to Peering connections and click Create peering connection. Name it **a-b-peer**, select vpc-a as the Requester, and vpc-b as the Accepter in your account. Click Create peering connection.
- **Accept Connection:** Select the newly created peering connection, click Actions, and choose Accept request.
- **Update Private Route Table A:** Navigate back to Route tables. Select rt-a-private, go to the Routes tab, click Edit routes, add a destination of 10.20.0.0/16, select Peering Connection as the target, choose your pcx-ID, and click Save changes.
- **Update Private Route Table B:** Select rt-b-private, click Edit routes, add a destination of 10.10.0.0/16, targeting the same Peering Connection pcx-ID, and save.
- **Update Public Route Table:** Select rt-a-public, click Edit routes, add a destination of 10.20.0.0/16, targeting the same Peering Connection pcx-ID (this is necessary for the Bastion to securely access App-B), and save.

Route Table	Destination	Target
rt-a-public	10.10.0.0/16	local
rt-a-public	0.0.0.0/0	igw-a
rt-a-public	10.20.0.0/16	pcx-ID (a-b-peer)
rt-a-private	10.10.0.0/16	local
rt-a-private	10.20.0.0/16	pcx-ID (a-b-peer)
rt-b-private	10.20.0.0/16	local
rt-b-private	10.10.0.0/16	pcx-ID (a-b-peer)

Step 6: Verification

- **Forward Path Test:** SSH into App-A and run a ping to App-B to verify 4/4 successful ICMP pings.
- **Reverse Path Test:** SSH into App-B and run a ping to App-A to verify 4/4 successful ICMP pings.
- **Private SSH Test:** Open a terminal on your local machine and successfully establish a direct SSH connection to App-B using the Bastion as a proxy.
- **Isolation Test:** In the EC2 console, verify that neither App-A nor App-B possesses a public IPv4 address.
- **Controlled Failure:** Navigate to Security Groups, select sg-app-b, edit the inbound rules to temporarily remove the ICMP rule, and run a ping from App-A to prove the security groups actively govern the traffic. Re-add the rule to restore connectivity.

4. Clean up, Risk Management, Billing and Cost

Billing and Cost Considerations

- Plan costs prior to launch. Resources such as EC2 instances, EBS volumes, and public IPv4 addresses incur AWS charges. Ensure sufficient account credits and check current pricing.
- The architecture explicitly avoids utilizing a NAT gateway, optimizing for lower lab cost.
- Ensure to keep resources active only for the duration of the project and execute cleanup immediately upon completion.

Risk Management and Troubleshooting

- **Architectural Constraints:** Ensure VPC CIDR blocks do not overlap; overlapping secondary CIDRs will prevent peering creation.
- **Concept Awareness:** Peering is one-to-one and non-transitive. Creating peering doesn't alter route tables automatically; explicit forward and return routes are strictly necessary.
- **Security Risks:** Never upload or paste private key files directly onto EC2 instances. Scope all security group rules to /32 granular IPs where possible instead of 0.0.0.0/0.
- **Troubleshooting Protocol:** If SSH or Ping fails, logically verify instance state, correct destination IP, verify the subnet's explicit route table associations, and evaluate exact security group parameters in that order.

Clean Up Steps

- **Compute & Credentials:** Terminate bastion, app-a, and app-b instances in the EC2 console. Verify EBS lab volumes are deleted. Release any Elastic IPs if used. Delete the lab-key key pair.
- **Peering & Network Dependencies:** Go to the VPC console and delete the peering connection (a-b-peer), which will also automatically remove the associated peer routes. Delete the lab security groups once network interfaces terminate. Remove subnet associations, delete subnets, custom route tables, and the internet gateway.
- **Final Verification:** Delete both vpc-a and vpc-b. Filter resources by project tags to ensure all infrastructure is removed. Verify the billing dashboard to ensure no lingering chargeable resources exist.